



Roteiro — Vídeo de Demonstração

Framework para Sistemas de Formatura Online (Microservices + Componentes de Software)

Disciplina: ECOM189 — Reuso de Software e Metodologias Ágeis · **Autor:** Marcos **Duração-alvo:** 6–7 minutos **Telas usadas:** VS Code · navegador (<http://localhost:5174>) · terminal do Docker · PDF do relatório/diagramas



Checklist antes de gravar

- ☐ Subir o backend: `docker compose up -d` (conferir todos "Up" com `docker compose ps`)
 - ☐ Subir o frontend: `npm run dev` na pasta `frontend` (anotar a porta: 5173 ou 5174)
 - ☐ Abrir no VS Code, em abas, os arquivos: `MeioPagamento.java` , `PagamentoPix.java` , `PagamentoBoleto.java` , `EstrategiaRateio.java` , `GestorFormatura.java` , `DemoFramework.java`
 - ☐ Abrir o diagrama de componentes (`Figura 1` do relatório) e o de hot spots
 - ☐ Fechar abas/notificações que possam atrapalhar a gravação
-

Bloco 1 — Abertura (0:00 – 0:30)

Tela: capa do relatório ou slide com o título.

"Olá, professor. Sou o Marcos. Este é o projeto da disciplina de Reuso de Software: **um framework para sistemas de formatura online, baseado em microservices e componentes de software**. Vou mostrar primeiro o conceito, depois o código do framework e, por fim, a aplicação rodando."

Bloco 2 — O conceito de framework (0:30 – 1:45)

Tela: diagrama parte invariante × variante / slide de hot spots.

"O ponto central é que isto **não é um aplicativo de formatura — é um framework**. Ele captura a **parte invariante** comum a qualquer formatura, cerca de 80% da lógica, e expõe **pontos de extensão, os hot spots**, que cada curso customiza."

"Usei os padrões **Template Method** e **Strategy**: o framework define o esqueleto fixo — os **frozen spots** — e chama o código da aplicação nos hot spots. Isso é a **inversão de controle**: é o framework que chama a aplicação, e não o contrário."

"O requisito era no mínimo 4 hot spots; o meu tem **6**: rateio, pagamento, evento, votação, notificação e pacotes."

Bloco 3 — Arquitetura (1:45 – 2:45)

Tela: Figura 1 do relatório (diagrama de componentes).

"A arquitetura é de **microserviços**, com um **API Gateway** como porta única de entrada, roteando para sete serviços: autenticação, turmas, financeiro, eventos, comissão e notificação."

"O detalhe de reuso é este: todos os microserviços **dependem do mesmo componente**, o `framework-core` — onde ficam os hot spots. Esse é o reuso de software no nível arquitetural: o framework é um componente compartilhado e versionado."

Bloco 4 — O código do framework (2:45 – 4:15)

Tela: VS Code.

4.1 — Hot spot Template Method (`MeioPagamento.java`)

"Aqui está o hot spot de pagamento. O método `processar` é `final` — é o **frozen spot**: ele valida, cobra e gera o recibo, sempre nessa ordem. O passo variável, `executarCobranca`, é **abstrato** — é o hot spot."

(*abrir `PagamentoPix` e `PagamentoBoleto`*) "Cada meio de pagamento só implementa o passo variável. O Boleto ainda sobrescreve a validação para exigir valor mínimo."

4.2 — Hot spot Strategy (`EstrategiaRateio.java`)

"O rateio usa Strategy: uma interface com três implementações — igualitário, por adesão e proporcional à renda. Trocar a estratégia muda o comportamento sem tocar no framework."

4.3 — Parte invariante e prova do reuso (`GestorFormatura.java` , `DemoFramework.java`)

"O `GestorFormatura` é a parte invariante: orquestra os hot spots por composição. E aqui na `DemoFramework` está a prova: **duas formaturas diferentes — Medicina e Engenharia — construídas a partir do mesmo núcleo**, só plugando hot spots diferentes. É exatamente o exemplo de framework que vimos em aula."

Bloco 5 — Demonstração ao vivo (4:15 – 6:15)

★ *parte mais importante*

Tela: terminal (containers) → navegador (`localhost:5174`).

"Vou subir tudo com **Docker** — um comando, `docker compose up` , sobe os sete serviços." (*mostrar `docker compose ps` com todos "Up"*)

Exercite os 6 cards narrando. **Truque:** mostre o mesmo hot spot dando resultados diferentes ao trocar a implementação.

- **Rateio:** "Com a estratégia **igualitária** e R\$ 30.000, todos pagam R\$ 10.000. Agora troco para **proporcional à renda** — os valores mudam conforme a renda. Mesma operação, hot spot diferente."
- **Pagamento:** "Pago via Pix — recebo o comprovante. Troco para Boleto — comprovante diferente, mesmo fluxo."
- **Evento:** "Organizo a Colação de Grau; depois o Baile — custo e etapas são calculados de forma diferente, porque cada evento implementa o hot spot do seu jeito."
- **Votação:** "Apuro por maioria simples; depois por quórum qualificado — a regra muda o resultado."

- **Notificação:** "Envio por e-mail, depois por WhatsApp."
- **Pacotes:** "A fábrica lista os pacotes disponíveis — foto, vídeo e álbum."

Bloco 6 — Testes e encerramento (6:15 – 7:00)

Tela: `FrameworkTest.java` ou a tabela de testes do relatório (seção 10).

"Por fim, o framework tem **testes JUnit** validando os hot spots, e validei a integração ponta a ponta pelo gateway."

"Resumindo: um framework de domínio com **6 hot spots**, padrões Template Method, Strategy e Factory, arquitetura de microservices e um componente reutilizável compartilhado — demonstrando, na prática, frozen spots, hot spots e inversão de controle. Obrigado!"



Dicas de gravação

- **Deixe tudo aberto antes de gravar:** containers no ar, frontend na porta, arquivos `.java` já abertos em abas.
- **Ritmo:** não leia código linha a linha — aponte só o `final` (frozen) e o `abstract` (hot spot) em cada hotspot.
- **Momento-chave:** trocar a estratégia de rateio e ver o número mudar ao vivo. Não pule — é a prova de reuso + variação.
- Mostre o `docker compose ps` com todos "Up" — evidencia os microservices reais.
- Fale o nome dos **padrões** (Template Method, Strategy, Factory) — é vocabulário que pontua.
- Grave em uma resolução em que o código fique legível (zoom da fonte no VS Code em ~16–18pt).